



(/rol/app/)

Menu

Korean

[홈 \(/rol/app/\) \(home\)](#) > [Managing Traffic with OpenShift Service Mesh](#)> [Chapter 1: Managing Traffic with OpenShift Service Mesh](#)

Managing Traffic with OpenShift Service Mesh

Lesson

Lab Environment

Traffic Routing and Splitting with OpenShift Service Mesh

Objectives

- Explain how DestinationRule subsets enable routing to specific service versions.
- Describe weighted, header-based, and path-based routing techniques and their use cases.
- Outline a progressive canary deployment strategy that minimizes risk during rollouts.

Traffic Routing and Splitting with OpenShift Service Mesh

Red Hat OpenShift Service Mesh provides powerful traffic routing capabilities that control how requests flow to different versions of your services. You can implement routing strategies by using virtual services and destination rules without modifying application code.

Traffic routing is essential for modern deployment strategies such as canary releases, A/B testing, and blue-green deployments. These strategies enable you to release new versions gradually, test features with specific user groups, and reduce the risk of widespread failures.

OpenShift Service Mesh provides three core routing techniques:

Weighted routing

Distributes traffic across service versions based on percentage allocations, enabling controlled rollouts and gradual migrations.



Header-based routing

< 이전

4/15

다음 >



Routes traffic based on HTTP request headers, supporting user segmentation and feature flags.

Path-based routing

Routes traffic based on the request URI path, enabling API versioning and endpoint-specific routing.

Destination Rules and Service Subsets

Before implementing traffic routing, you must understand how destination rules define service subsets.

A *subset* represents a specific version of a service, identified by labels on service instances. Destination rules define these subsets, enabling virtual services to route traffic to specific service versions. A destination rule resource defines subsets in the `spec.subsets` field. Each subset has a name and a set of OpenShift labels that identify which service instances belong to that subset.

Consider a `reviews` service with three versions deployed:

- `reviews-v1`: Service instances with label `version: v1`
- `reviews-v2`: Service instances with label `version: v2`
- `reviews-v3`: Service instances with label `version: v3`

The following destination rule defines three subsets for the `reviews` service:

```
apiVersion: networking.istio.io/v1
kind: DestinationRule
metadata:
  name: reviews
spec:
  host: reviews 1
  subsets: 2
    - name: v1 3
      labels: 4
        version: v1
    - name: v2
      labels:
        version: v2
    - name: v3
      labels:
        version: v3
```

- ¹ The Kubernetes service name to which this destination rule applies.
- ² List of subsets for this service.
- ³ Subset name referenced by virtual services.
- ⁴ Labels that identify service instances belonging to this subset.

After defining subsets in a destination rule, you can reference them in virtual service routing rules to direct traffic to specific service versions.

IMPORTANT

Always create destination rules before creating virtual services that reference subsets. If a virtual service references a non-existent subset, then traffic routing fails.

Weighted Routing

Weighted routing distributes traffic across service versions based on percentage allocations. This technique supports progressive rollouts, where you gradually shift traffic from a stable version to a new version.

Unlike random Kubernetes load balancing, weighted routing provides deterministic control over traffic distribution. You specify exact percentages, and OpenShift Service Mesh ensures that traffic is distributed according to those weights.

Virtual services implement weighted routing with the `weight` field in route destinations. The weights can sum to any value, but typically sum to 100 so that they represent a percentage distribution. The following image shows weighted routing with distributed traffic with an 80/15/5 split:

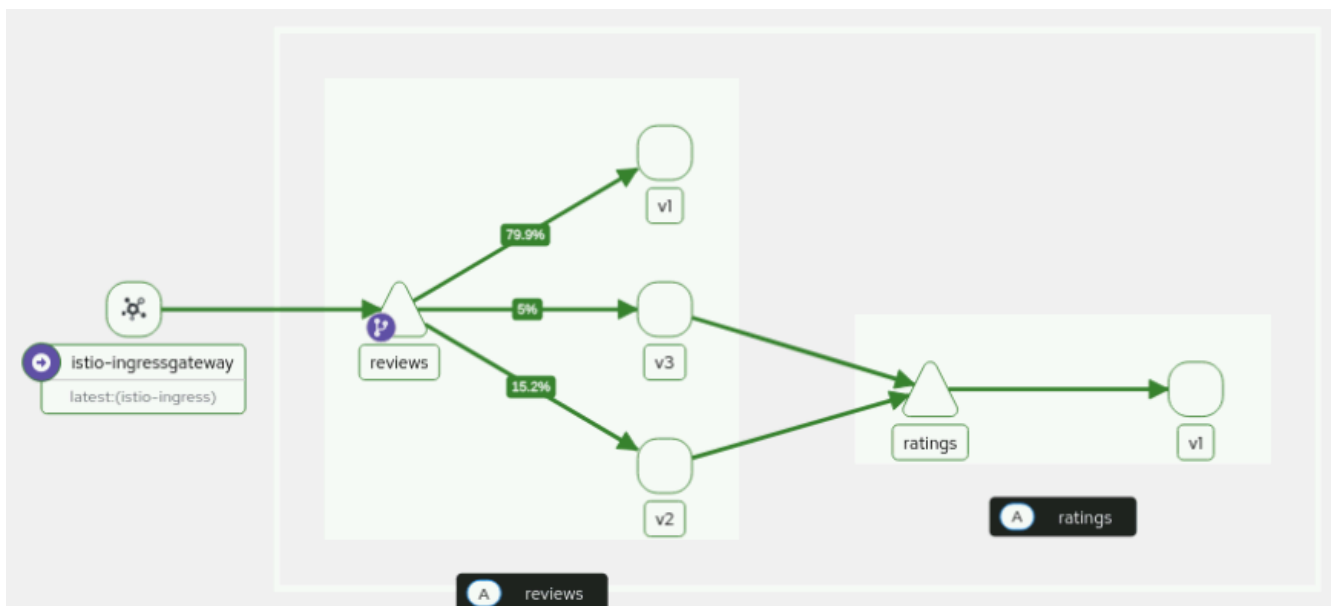


Figure 1.4: Weighted routing distributes traffic by percentage (80/15/5 split)

The following example distributes traffic with an 80/15/5 split:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route: ❶
    - destination: ❷
        host: reviews
        subset: v1
        weight: 80 ❸
    - destination:
        host: reviews
        subset: v2
        weight: 15
    - destination:
        host: reviews
        subset: v3
        weight: 5
```

- ❶ List of destinations for this route.
- ❷ First destination: the v1 subset of the reviews service.
- ❸ Weight (percentage) of traffic to send to this destination.

Weighted routing assists with the following deployment strategies:

Canary deployments

Gradually increase traffic to a new version while monitoring for issues. Start with 5-10% and progressively increase as confidence grows.

A/B testing

Split traffic between two versions to compare performance, user engagement, or conversion rates.

Blue-green deployments

Maintain two complete production environments and switch traffic between them.

Load testing

Direct a percentage of production traffic to a new version to validate performance under real-world conditions.

Header-based Routing

Header-based routing directs traffic to different service versions based on HTTP request headers. This technique enables user segmentation, feature flags, and API client differentiation without application code changes.

OpenShift Service Mesh inspects HTTP headers and matches them against routing rules. When a match occurs, traffic routes to the specified destination. This happens transparently at the mesh level, independent of application logic.

Virtual services implement header-based routing by using the match field with headers conditions. The match field supports exact matching, prefix matching, and regex matching. The following diagram illustrates how OpenShift Service Mesh routes traffic based on the `x-user-type` header:

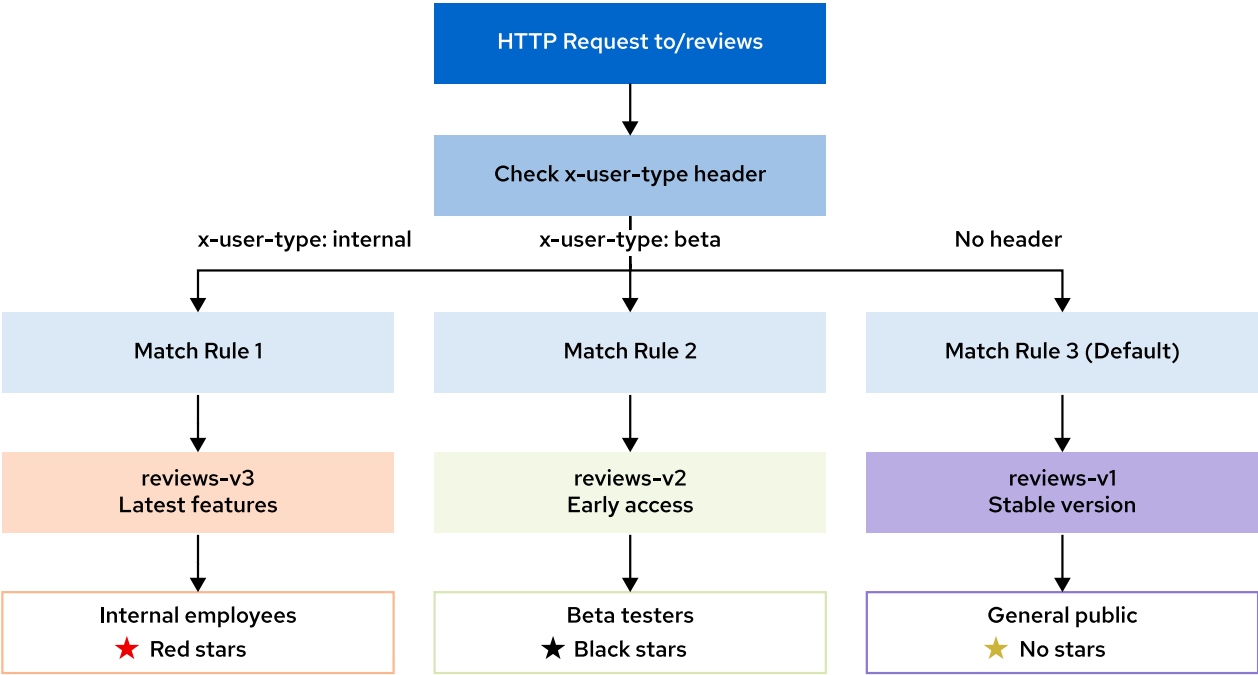


Figure 1.5: Header-based routing directs traffic based on HTTP request headers

The following example routes traffic based on the `x-user-type` header:

```

apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: ❶
    - uri:
        prefix: /reviews
      headers: ❷
        x-user-type: ❸
          exact: internal ❹
    route:
    - destination:
        host: reviews
        subset: v3 ❺
  - match: ❻
    - uri:
        prefix: /reviews
      headers:
        x-user-type:
          exact: beta
    route:
    - destination:
        host: reviews
        subset: v2
  - match: ❼
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1

```

- ❶ First routing rule with match conditions.
- ❷ Header match conditions.
- ❸ Name of the HTTP header to inspect.
- ❹ StringMatch rule: matches when the header value equals `internal`.
- ❺ Route requests with `x-user-type: internal` to the `v3` subset.
- ❻ Second routing rule for beta testers.
- ❼ Default route for all other requests (no header match required).

IMPORTANT

Routing rules are evaluated in order from top to bottom. Always place more specific rules before general rules. The first matching rule determines the routing destination.

OpenShift Service Mesh supports the following header matching strategies:

Exact match

The header value must exactly match the specified value.

```
headers:  
  end-user:  
    exact: redhatter
```

Prefix match

The header value must start with the specified prefix.

```
headers:  
  user-agent:  
    prefix: Mozilla
```

Regex match

The header value must match the specified regular expression.

```
headers:  
  cookie:  
    regex: "^(.*?;)?(session=[^;]+)(;.*)?$"
```

Header-based routing enables the following advanced scenarios:

Feature flags

Control feature visibility by routing specific user groups to different service versions. For example, route requests with a feature flag header to a version with experimental features.

API versioning

Route requests based on API version headers without changing URLs. Clients specify their desired API version in an `API-Version` header.

Internal and external users

Route internal employee traffic to staging versions and external traffic to production. Identify internal users by inspecting authentication headers or custom headers.

Mobile and web clients

Route mobile app traffic differently from web traffic by inspecting the `User-Agent` header. This optimizes responses for different client types.

Path-based Routing

Path-based routing directs traffic to different service versions based on the request URI path. This technique is essential for API versioning, where different endpoints represent different API versions.

OpenShift Service Mesh inspects the request URI and routes traffic to the appropriate backend based on path matching rules. The mesh can also rewrite URIs before forwarding requests to backend services.

Virtual services implement path-based routing by using the `match` field with `uri` conditions. The following example implements API versioning by routing different URI paths to different service versions:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: ❶
    - uri:
        prefix: /api/v3/reviews ❷
      rewrite: ❸
        uri: /reviews
      route:
      - destination:
          host: reviews
          subset: v3 ❹
    - match:
      - uri:
          prefix: /api/v2/reviews
        rewrite:
          uri: /reviews
        route:
        - destination:
            host: reviews
            subset: v2
    - match: ❺
      - uri:
          prefix: /reviews
        route:
        - destination:
            host: reviews
            subset: v1
```

❶ First routing rule: matches requests to `/api/v3/reviews`.

❷ URI prefix match condition.

❸ Rewrite the URI before forwarding to the backend. The backend service only accepts `/reviews` endpoints, not `/api/v3/reviews`.

- 4 Route to the v3 subset after rewriting the URI.
- 5 Default route for standard /reviews paths (no rewrite needed).

This configuration results in the following rules:

- Requests to /api/v3/reviews/* are rewritten to /reviews/* and routed to reviews-v3
- Requests to /api/v2/reviews/* are rewritten to /reviews/* and routed to reviews-v2
- Requests to /reviews/* route to reviews-v1 without URI rewriting

URI Rewriting

URI rewriting is essential when the external API structure differs from the internal service structure. The `rewrite` field in a virtual service modifies the request URI before forwarding it to the backend service.

For example, clients request /api/v3/reviews/1, but the backend service expects /reviews/1. The virtual service rewrites /api/v3/reviews/1 to /reviews/1 before forwarding the request.

This separation enables the following characteristics:

- Present a versioned API to clients
- Maintain a consistent internal service interface
- Change external API paths without modifying backend services
- Support multiple API versions pointing to the same backend

OpenShift Service Mesh supports the following URI matching strategies:

Prefix match

The URI must start with the specified prefix.

```
uri:  
  prefix: /api/v2
```

Exact match

The URI must exactly match the specified value.

```
uri:  
  exact: /health
```

Regex match

The URI must match the specified regular expression.

```
uri:  
  regex: "^/api/v[0-9]+/.*$"
```

Use Cases for Path-based Routing

Path-based routing supports the following scenarios:

API versioning

Expose multiple API versions simultaneously without duplicating backend services. For example, you can route `/api/v1/*`, `/api/v2/*`, and `/api/v3/*` to different service versions.

Service decomposition

Route different URI paths to different microservices. For example, you can route `/catalog/*` to the catalog service and `/cart/*` to the cart service.

Gradual API migration

Maintain legacy endpoints and introduce new API structures. Route old paths to legacy services and new paths to modernized services.

Environment-specific routing

Route specific paths to staging or development versions. For example, you can route `/beta/*` to a staging environment.

Progressive Canary Deployments

A canary deployment is a deployment strategy that gradually shifts traffic from a stable version to a new version. You start by routing a small percentage of traffic to the new version, monitor its behavior, and progressively increase the traffic percentage as confidence grows.

This approach minimizes risk by limiting the impact of potential issues in the new version. If problems occur, then you can quickly roll back by reducing traffic to the new version or setting it to zero. A progressive canary deployment typically follows these phases:

Phase	Traffic Distribution	Purpose
Initial deployment	95% stable, 5% canary	Detect critical issues with minimal user impact
Increased exposure	75% stable, 25% canary	Validate stability under moderate load
Balanced rollout	50% stable, 50% canary	Confirm performance at scale
Full deployment	0% stable, 100% canary	Complete the migration

The following example shows an initial canary deployment with 5% traffic to the new version:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match:
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1
      weight: 95 ❶
    - destination:
        host: reviews
        subset: v3
      weight: 5 ❷
```

❶ Maintain 95% of traffic on the stable version (v1).

❷ Direct only 5% of traffic to the new canary version (v3).

To progress the canary deployment, increase the canary version's weight and consider the following:

Start small

Begin with 5-10% canary traffic to minimize risk. This limits the impact of potential issues.

Monitor metrics

Track error rates, latency, and other key performance indicators for both versions. In production environments, monitor error rates, latency, and throughput by using OpenShift Service Mesh observability tools, such as Kiali and Prometheus, between each traffic increase. Wait for metrics to stabilize before progressing to the next phase.

Progress gradually

Increase traffic in increments: 5%, 25%, 50%, and 100%. Wait between increases to observe system behavior under each configuration.

Combine with header-based routing

Route internal users or beta testers to the canary version regardless of percentage. This provides additional validation before exposing the new version to all users.

Prepare for rollback

Keep configuration for immediate rollback (set canary weight to 0). Monitor alerting systems to detect issues quickly.

Combining Canary with Header-based Routing

A sophisticated canary strategy combines weighted routing with header-based routing. This enables you to route specific user groups (internal employees, beta testers) to the canary version. You can gradually increase the percentage for general users.

The following example demonstrates this approach:

```
apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: ❶
    - uri:
        prefix: /reviews
        headers:
          x-user-type:
            exact: internal
      route:
      - destination:
          host: reviews
          subset: v3 ❷
  - match: ❸
    - uri:
        prefix: /reviews
      route:
      - destination:
          host: reviews
          subset: v1
          weight: 90
      - destination:
          host: reviews
          subset: v3
          weight: 10 ❹
```

- ❶ First rule: route internal users to the canary version.
- ❷ Internal users always get the canary version (v3).
- ❸ Second rule: general users get a weighted distribution.
- ❹ General users get 10% canary traffic.

This configuration ensures that internal users validate the canary version before increasing exposure to general users.

Routing Rule Precedence

OpenShift Service Mesh evaluates routing rules in order from top to bottom. The first rule that matches a request determines the routing destination.

Understanding rule precedence is critical for configuring complex routing scenarios.

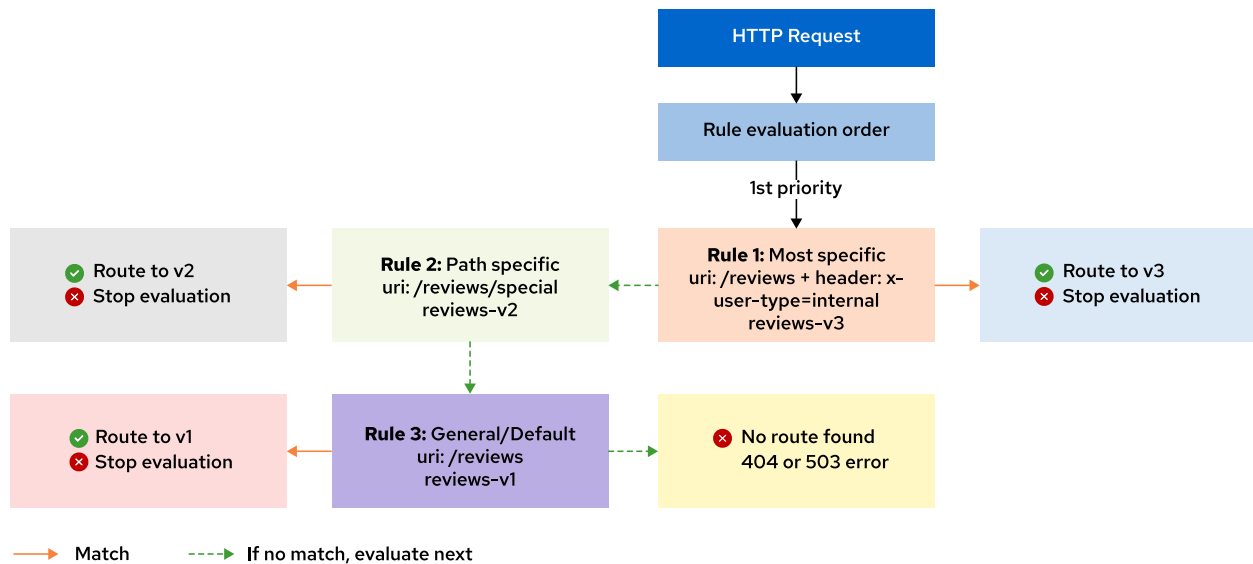


Figure 1.6: Virtual services evaluate routing rules in sequential order with first-match-wins behavior

Virtual services evaluate routing rules by using the following process:

1. OpenShift Service Mesh inspects the incoming request (URI, headers, etc.)
2. Rules are evaluated sequentially from top to bottom
3. The first rule with matching conditions is selected
4. Traffic routes to the destination specified in the matching rule
5. Subsequent rules are ignored

The following example demonstrates rule precedence:

```

apiVersion: networking.istio.io/v1
kind: VirtualService
metadata:
  name: reviews
spec:
  hosts:
  - "*"
  gateways:
  - reviews-gateway
  http:
  - match: ❶
    - uri:
        prefix: /reviews
      headers:
        x-user-type:
          exact: internal
    route:
    - destination:
        host: reviews
        subset: v3
  - match: ❷
    - uri:
        prefix: /reviews/special
    route:
    - destination:
        host: reviews
        subset: v2
  - match: ❸
    - uri:
        prefix: /reviews
    route:
    - destination:
        host: reviews
        subset: v1

```

- ❶ Most specific rule: matches `/reviews` with internal user header.
- ❷ Path-specific rule: matches `/reviews/special`.
- ❸ General rule: matches all `/reviews` requests.

IMPORTANT

Always place more specific rules before general rules. If rule 3 (general `/reviews` match) appears before rule 2 (specific `/reviews/special`), then the specific rule never executes because the general rule matches first.

Avoiding Routing Errors

When implementing traffic routing, avoid these errors:

Overlapping routing rules

Multiple rules can match the same request, but only the first matching rule executes. Carefully order the rules so that the correct rule matches first.

Missing destination rule

Referencing a subset in a virtual service without defining it in a destination rule causes routing failures. Always create destination rules before virtual services that reference subsets.

Incorrect weight calculations

Weights that do not sum to 100 still work, but can cause unexpected traffic distribution. For clarity, always use weights that sum to 100.

Forgetting URI rewrites

When routing to backends with different URI structures, forgetting to rewrite the URIs results in 404 errors. Use the `rewrite` field to transform URIs before forwarding requests.

Testing without observability

Implementing routing without monitoring makes it difficult to verify correct behavior. Always use observability tools to validate routing configurations.

REFERENCES

Istio.io: Traffic Management Concepts

(<https://istio.io/v1.26/docs/concepts/traffic-management/>)

Istio.io: Virtual Service

(<https://istio.io/v1.26/docs/reference/config/networking/virtual-service/>)

Istio.io: Destination Rule

(<https://istio.io/v1.26/docs/reference/config/networking/destination-rule/>)

Istio.io: Traffic Management Best Practices

(<https://istio.io/v1.26/docs/ops/best-practices/traffic-management/>)

Istio.io: Traffic Shifting Task (<https://istio.io/v1.26/docs/tasks/traffic-management/traffic-shifting/>)

Revision: ad00081-3.1-ed4-20260310-8ae9760



이용 약관 (<https://www.redhat.com/ko/about/terms-use>)

모든 정책 및 지침

(<https://www.redhat.com/ko/about/all-policies-guidelines>)

릴리스 노트

쿠키 기본설정

(https://docs.redhat.com/en/documentation/red_hat_learning_subscription/1-latest/html-single/red_hat_learning_subscription_release_notes/index)